

FONDAMENTI DI PROGRAMMAZIONE WEB

LABORATORIO

Capitolo 8 – Option API e Server

Argomenti del giorno

- ▶ Vue:
 - ▶ Option API – struttura contenuto `<script> </script>`
 - ▶ Interpolazione del testo
 - ▶ Comunicazione componente Padre -> componente Figlio
 - ▶ Collegamento al Server
 - ▶ Direttive v-for e v-if

Option API

- ▶ Si definisce la logica di un componente utilizzando un oggetto contenente diverse proprietà. Qui ne vedete solo alcune (data, methods e mounted).
- ▶ Le proprietà restituite da data() diventano parte dello stato reattivo e saranno accessibili tramite `this`.
- ▶ I metodi sono funzioni che modificano lo stato e attivano gli aggiornamenti. Possono essere usati come gestori di eventi nei template (tramite v-on o @).
- ▶ I metodi del ciclo di vita sono chiamati in diverse fasi del ciclo di vita di un componente. Mounted() viene chiamata quando il componente viene 'montato' nel DOM.

```
<script>
export default {
  data() {
    return {
      count: 0
    }
  },
  methods: {
    increment() {
      this.count++
    }
  },
  mounted() {
    console.log(`The initial count is ${this.count}.`)
  }
}
</script>

<template>
  <button @click="increment">Count is: {{ count }}</button>
</template>
```

Esercizi lezione precedente

- ▶ Vi è stato chiesto di creare le componenti di recensioni, news e utenti.
- ▶ La struttura del componente recensioni può essere questa:

```
<script>
</script>
<template>
  <article class="col-6 review-card">
    <h3 class="review-title">Title</h3>
    <p class="review-content">
      Lorem ipsum dolor sit amet consectetur adipiscing elit. Quisque faucibus ex sapien vitae
      pellentesque sem placerat. In id cursus mi pretium tellus duis convallis. Tempus leo eu
      aenean sed diam urna tempor. Pulvinar vivamus fringilla lacus nec metus bibendum egestas.
      Iaculis massa nisl malesuada lacinia integer nunc posuere. Ut hendrerit semper vel class
      aptent taciti sociosqu. Ad litora torquent per conubia nostra inceptos himenaeos.
    </p>
    <p><b>Giudizio:</b> - <b>Likes:</b> - <b>Dislikes:</b></p>
  </article>
</template>
```

- ▶ L'idea di base di un componente è quella di poterlo riutilizzare in svariate parti del codice, rendendo il codice più pulito e manutenibile, perciò la struttura deve essere precisa ma il contenuto deve essere generalizzabile.

Interpolazione del testo

- L'interpolazione del testo ci permette di rendere dinamiche le **stringhe** presenti template. Queste infatti assumeranno il valore contenuto nella variabile ricollegata presente in data.

ReviewCard.vue

```
<script>
  export default{
    data(){
      return {
        titolo: "",
        commento: "",
        voto: 0,
        num_like: 0,
        num_dislike:0,
      }
    }
  }
</script>
```

```
<template>
  <article class="col-6 review-card">
    <h3 class="review-title">{{ titolo }}</h3>

    <p class="review-content">
      {{ commento }}
    </p>

    <p><b>Giudizio: {{ voto }}</b> - <b>Likes: {{
num_like }}</b> - <b>Dislikes: {{ num_dislike }}</b></p>
  </article>
</template>
```

Comunicazione tra Padre e Figlio

- ▶ Tuttavia in un sito normalmente si hanno diversi componenti annidati.
- ▶ Nel nostro sito abbiamo ad esempio **HomeView.vue** (*componente padre*) che contiene **ReviewCard** (*componente figlio*).
 - ▶ Questi "parlano tra di loro".
- ▶ I componenti padri comunicano con i figli (Padre -> Figlio) attraverso le **proprietà**.
- ▶ I componenti figli comunicano con i componenti padri (Figlio-> Padre) attraverso gli **eventi**.

Comunicazione tra Padre e Figlio

- ▶ Andiamo a modificare il codice del componente Figlio **ReviewCard.vue** utilizzando l'interpolazione del testo.
 - ▶ Il template rimane come era prima.
 - ▶ Quello che cambia è il codice di script.

```
<template>
  <article class="col-6 review-card">
    <h3 class="review-title">{{ titolo }}</h3>

    <p class="review-content">
      {{ commento }}
    </p>

    <p><b>Giudizio: {{ voto }}</b> - <b>Likes: {{ num_like
  }}</b> - <b>Dislikes: {{ num_dislike }}</b></p>
    </article>
  </template>
```

- ▶ In questo modo popoleremo il **template** con il contenuto delle variabili presenti nello script in **props**.
- ▶ Le **props** sono **proprietà** che il componente riceverà in input dal componente Padre.

ReviewCard.vue

```
<script>
  export default{
    props:{
      titolo: {
        type: String,
        required: true
      },
      commento: {
        type: String,
        required: true
      },
      voto: {
        type: Number,
        required: true
      },
      num_like: {
        type: Number,
        default: 0
      },
      num_dislike: {
        type: Number,
        default: 0
      }
    }
  }
</script>
```

Comunicazione tra Padre e Figlio

- Ora modifichiamo il codice del componente Padre (ovvero **HomeView.vue**).

HomeView.vue

```
<script>
  import ReviewCard from '@components/ReviewCard.vue';
  export default {
    components: { ReviewCard },
    data() {
      return {
        titolo: "Lady Gaga - A star is born",
        commento: "Mai visto film più eccezionale. Non tanto per il pluripremiato
Bradley Cooper ma per la fantasmagorica e bellissima Lady Gaga che si afferma un'attrice nata.
Mai visto un film con un'attrice così nella parte! Avrebbe dovuto vincere l'oscar come
protagonista :(",
        voto: 5, num_like: 254, num_dislike: 1023
      }
    }
  }
</script>
```

- Nello script importiamo il componente e definiamo ciò che vogliamo passare a **ReviewCard.vue**.
- In questo caso le variabili si chiamano nello stesso modo dei props, anche se non è necessario.

Comunicazione tra Padre e Figlio v-bind

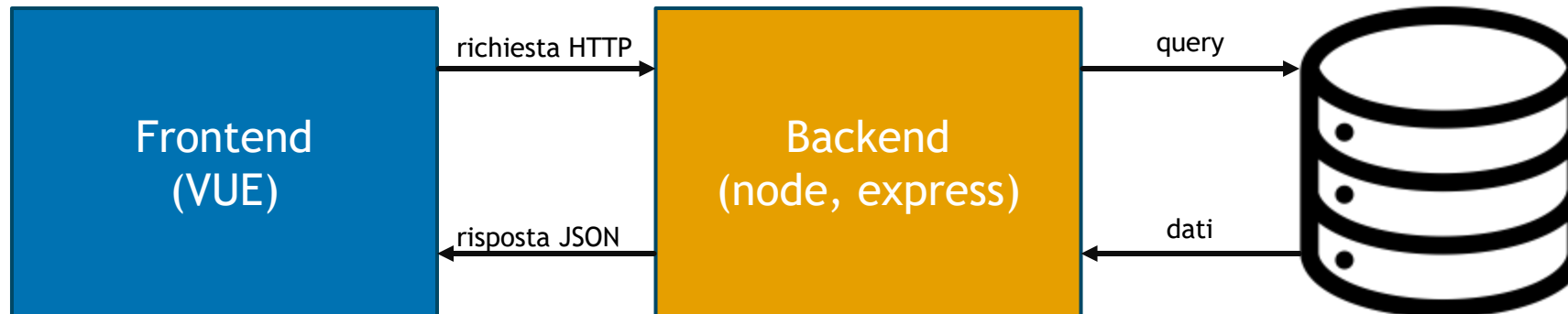
- ▶ Nel **template** di **HomeView.vue** richiamiamo il nostro componente e passiamo i dati alle props.
- ▶ Per farlo utilizziamo i ":" che sono la shortcut della direttiva **v-bind**, che permette di *collegare* dinamicamente i valori dei props al dato in input.

HomeView.vue

```
<article class="col-9">
  <section>
    <h2>Recensioni in evidenza</h2>
    <ReviewCard
      :titolo="titolo"
      :commento="commento"
      :voto="voto"
      :num_like="num_like"
      :num_dislike="num_dislike">
    </ReviewCard>
  </section>
```

Collegamento frontend-backend

- ▶ Rendiamo veramente **dinamico** il contenuto del nostro sito.
- ▶ Facciamo in modo che il **frontend** possa recuperare/salvare dei dati nel **DB**.
- ▶ Per farlo useremo gli endpoint definiti nel **backend**.



Collegamento frontend-backend

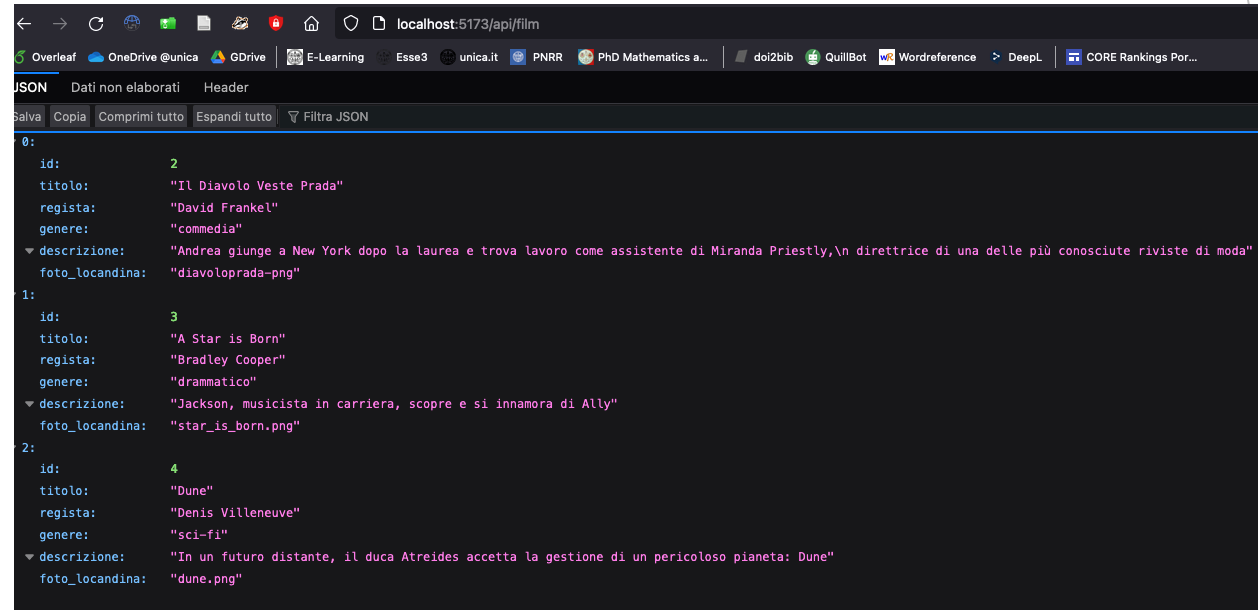
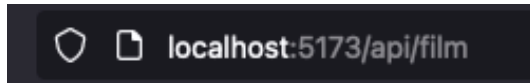
- ▶ **host:** l'indirizzo su cui l'applicazione web è in ascolto
 - ▶ Perché localhost? Perché stiamo lavorando in locale.
- ▶ **port:** la porta su cui il nostro sito, l'app VUE, rimane in ascolto.
 - ▶ Se non avete cambiato nulla, di default è la 5173
- ▶ **proxy:** qui definiamo le regole per inoltrare le chiamate al backend. In particolare, nel nostro caso, andiamo a intercettare tutte le richieste che iniziano con **/api/**. I parametri che la compongono sono:
 - ▶ **target:** l'indirizzo a cui girare la richiesta, ovvero il path del nostro server express (http://localhost:3000).
 - ▶ **secure:** un booleano che determina se accettare le richieste in HTTP o in HTTPS (con false non vengono accettate le chiamate in HTTPS).
 - ▶ **changeOrigin:** false, lascia invariato l'header Host.
 - ▶ **rewrite:** riscrive il path, in questo caso lo usiamo per rimuovere il prefisso **/api** prima di inoltrare (es. /api/users → /users).

vite.config.js

```
import { fileURLToPath, URL } from 'node:url'
import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'
import vueDevTools from 'vite-plugin-vue-devtools'
// https://vite.dev/config/
export default defineConfig({
  server: {
    host: 'localhost',
    port: 5173,
    proxy: {
      '/api/': {
        target: 'http://localhost:3000',
        secure: false,
        changeOrigin: false,
        rewrite: path => path.replace(/^\/api/, '')
      }
    }
  },
  plugins: [
    vue(),
    vueDevTools(),
  ],
  resolve: {
    alias: {
      '@': fileURLToPath(new URL('./src', import.meta.url))
    },
  },
})
```

Collegamento frontend-backend

- ▶ Se avviamo il **backend** (*npm start*) e ci colleghiamo all'URL **localhost:5173/api/film** otteniamo la lista di film contenuti nel nostro db. Stiamo passando per la **route** del server **getFilms**



Collegamento frontend-backend: modifiche alle queries

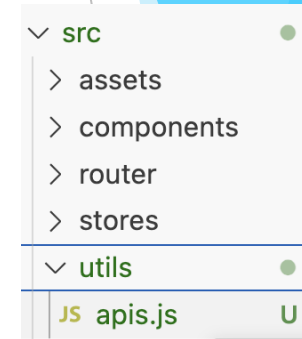
- ▶ In ***recensioni_queries.js*** dobbiamo recuperare anche il titolo del film. Infatti, la tabella recensioni **non contiene** il titolo del film a cui ci si riferisce ma solo l'id, perciò è necessario fare due **SELECT** e recuperare anche il titolo del film.
- ▶ Per farlo useremo due **SELECT**
 - ▶ Una prima **SELECT** con cui recuperare la recensione.
 - ▶ Una seconda **SELECT** con cui recuperare il nome del film.
 - ▶ Nel corso di Basi di Dati vedrete comandi più efficienti, come le JOIN.

recensioni_queries.js

```
const getRecensioni = 'SELECT  
recensioni.* FROM recensioni';  
const getRecensioneById = 'SELECT  
recensioni.* FROM recensioni WHERE  
film_id = $1';  
const getFilmTitleById = "SELECT  
titolo FROM film WHERE id = $1";
```

Collegamento frontend-backend: APIs

- ▶ Nella cartella **src** del nostro sito creiamo una cartella **utils** e all'interno creiamo un file **apis.js**
- ▶ In **apis.js** creiamo una funzione che farà la **chiamata** al nostro server e con cui recuperare una precisa recensione. Essendo una funzione **asincrona**, la sintassi per crearla e per esportarla è leggermente diversa.



CineVUE/src/utils/api.js

```
const getReviewByID = async (id) => {  
  const response = await fetch(`api/recensioni/${id}`);  
  return await response.json;  
}  
  
export {  
  getReviewByID,  
}
```

Collegamento frontend-backend: Recensioni in HomeVue

- ▶ In **HomeView.vue** settiamo le variabili, precedentemente passato alla **ReviewCard**, a dei valori di default.
- ▶ Importiamo le funzioni dal file **apis.js** e creiamo una funzione *asincrona* che recuperi una recensione e con l'oggetto ottenuto in risposta, **aggiorniamo** le nostre variabili.
- ▶ Tramite il metodo **mounted()** facciamo in modo che questa funzione venga eseguita **appena dopo la creazione del componente**.

HomeView.vue

```
<script>
import ReviewCard from '@components/ReviewCard.vue';
import * as Api from '@utils/apis.js';
export default {
  components: { ReviewCard },
  data() {
    return {
      titolo: "",
      commento: "",
      voto: 0,
      num_like: 0,
      num_dislike: 0
    }
  },
  methods: {
    async getReviewByID(id) {
      const data = await Api.getReviewByID(id);
      console.log(data);
      this.titolo = data.titolo;
      this.commento = data.commento;
      this.voto = data.voto;
      this.num_like = data.num_like;
      this.num_dislike = data.num_dislike;
    }
  },
  mounted() {
    this.getReviewByID(1);
  }
}
</script>
```

Collegamento frontend-backend: Recensioni in HomeVue

- ▶ Al momento abbiamo una funzione che ci restituisce una sola recensione, ma nella home abbiamo bisogno di tutte le recensioni. Dato che abbiamo già la query per farlo, in apis.js creiamo la funzione per eseguirla.

CineVUE/src/utils/api.js

```
const getReviews = async () => {  
  const response = await fetch('api/recensioni/');  
  return await response.json();  
}  
  
export {  
  getReviewByID,  
  getReviews  
}
```


Comunicazioni Client-Server: Recensioni in HomeVue

- ▶ In **HomeView.vue**, aggiorniamo lo script:
- ▶ Dato che stiamo ricevendo un **array** di recensioni, non serve tenere gli attributi singolarmente. Dichiariamo un array reviews **vuoto**.
- ▶ Nei metodi creiamo una nuova funzione **getReviews()** e assegniamo il risultato al vettore reviews.
- ▶ Bisogna anche aggiornare il metodo **mounted()** in quanto non dovrà più eseguire la funzione precedente.
 - ▶ Siete liberi di tenere la funzione **GetReviewsById()** nelle funzioni, ma fate in modo che non disturbi il funzionamento del sito.

HomeView.vue

```
<script>
  import ReviewCard from
    '@components/ReviewCard.vue';
  import * as Api from '@utils/apis.js';
  export default {
    components: { ReviewCard },
    data() {
      return {
        reviews: []
      }
    },
    methods: {
      async getReviews() {
        const data = await Api.getReviews();
        this.reviews = data;
      }
    },
    mounted() {
      this.getReviews();
    }
  }
</script>
```

Collegamento frontend-backend:

Direttive v-for e v-if

- ▶ Una direttiva in VUE è un elemento di sintassi che inizia con il prefisso **v-**.
- ▶ Le direttive **estendono** il comportamento degli attributi **HTML** standard e permettono di inserire parti della logica direttamente nel template.
- ▶ Oggi vedremo due direttive:
 - ▶ **v-for**: cicla su una collezione di dati e renderizza il markup corrispondente.
 - ▶ **v-if**: renderizza un elemento solo quando una condizione è vera.

Collegamento frontend-backend:

v-for

- ▶ Nel **template** in **HomeView.vue** utilizziamo la direttiva **v-for** per caricare nella pagina tutte le recensioni recuperate direttamente nel codice html.
- ▶ In questo modo **combinandola** al nostro componente, possiamo ottenere una **lista** di ReviewCard ognuna con valori **diversi**. Quindi otterremo lo stesso risultato del nostro sito statico ma questa volta stiamo recuperando i dati dal database.

HomeView.vue

```
<section>
  <h2>Recensioni in evidenza</h2>
  <ReviewCard v-for="review in reviews" :key="review.id" :titolo="review.titolo"
:commento="review.commento" :voto="review.voto"
  :num_like="review.num_like" :num_dislike="review.num_dislike">
    </ReviewCard>
</section>
```

Collegamento frontend-backend:

v-if

- Facciamo in modo che nella nostra **home** vengano mostrate solo recensioni che hanno un voto maggiore di 3. Possiamo utilizzare la direttiva **v-if**:

HomeView.vue

```
<section>
  <h2>Recensioni in evidenza</h2>
  <div v-for="review in reviews" :key="review.id">
    <div v-if="review.voto > 3">
      <ReviewCard :titolo="review.titolo" :commento="review.commento" :voto="review.voto"
        :num_like="review.num_like" :num_dislike="review.num_dislike">
      </ReviewCard>
    </div>
  </div>
</section>
```

- È importante sapere che la direttive **v-for** e **v-if** **non possono essere utilizzate nello stesso elemento html**. Perciò all'interno dell'article apriamo un div che conterrà solo la condizione.
 - Chiaramente esiste anche la controparte **v-else**, ma in questo caso non è servita.

Esercizi

- ▶ Aggiornate le pagine News e Chi siamo in modo da recuperare notizie e admin dal db

- ▶ Trovate un backup del db aggiornato e popolato su E-Learning
- ▶ Quando andrete a passare il path dell'immagine come prop, non potrete passarlo direttamente alla proprietà src dell'elemento img, Dovete creare una funzione che presa la stringa del prop, ne restituisca un URL col path dell'immagine da passare all'elemento

```
methods: {  
  getUrlImage() {  
    return new URL(`../assets/img/${this.imageName}`, import.meta.url);  
  }  
}
```

- ▶ Nella pagina News tramite le direttive v-if e v-else trovate il modo di dividere in notizie recenti e notizie vecchie
 - ▶ Per esempio, le notizia più vecchie di una certa data non dovranno essere mostrate nella sezione principale ma in quella apposita